Targeted Mining of Rare High-Utility Patterns

Peifeng Zhang[1], Jiahui Chen[1], Shicheng Wan[1], Wensheng Gan[2,3*]

[1]Guangdong University of Technology, Guangzhou 510006, China
[2]Jinan University, Guangzhou 510632, China
[3]Pazhou Lab, Guangzhou 510330, China
Email: {paddeyzhang, csjhchen, scwan1998, wsgan001}@gmail.com

***Abstract*—Pattern discovery has been widely studied and applied as a classical problem in data mining. As a subfield of itemset mining, identifying high-utility rare itemsets (HURI) can find abnormal but vital patterns in transaction databases. It plays a unique role in real-world scenarios such as anomaly detection and disease detection. However, with large-scale databases, the final results are often massive according to the user-specified threshold. In other words, the mining algorithm ignores the user's subjective interests and lacks interaction during the mining process. A pattern discovery algorithm may output many useless or uninteresting patterns. To this end, in this paper, we define the problem of mining targeted HURIs and propose a list-based algorithm called TaRP for effectively solving this issue. In addition, based on preliminary research, we propose several effective pruning strategies for improving the algorithm's performance. TaRP makes the results more interactive and specific by incorporating the user's prior knowledge during mining. It also has a natural performance advantage with the help of effective strategies. We also evaluated the proposed algorithm on several real-life datasets. The extensive experimental results demonstrate that TaRP not only correctly solves the problem but also has advantages in runtime and memory consumption, especially on dense datasets.**

***Index Terms*—targeted mining, rare pattern, high-utility itemset, utility-list**

I. Introduction

With the coming of the information age, data has gradually become more and more significant. Given a transaction database, many effective mining technologies reveal different kinds of knowledge from various metrics (e.g., frequency, confidence, and recency) [1], [2], [3]. The Apriori algorithm [4] first defined the association rule mining (ARM) task on transaction datasets. Inspired by the Apriori algorithm, researchers continuously improved pattern discovery algorithms to find interesting itemsets as effectively and efficiently as possible. Pattern discovery, such as frequent itemset mining (FIM) [5], [6] that only focuses on support metrics, is a classical subfield of data mining. Further research realized that the subjective properties of different items are also vital for enriching the value of results [7], [8]. To address this issue, the utility metric (e.g., profit, loss, and risk) is proposed.

High-utility itemset mining (HUIM) [9], [10], [11] differs from FIM in terms of metrics. FIM uses the total number of occurrences to measure the interest of different items, while HUIM takes the utility of an itemset as an interest evaluation. For example, the profitability of goods may be the primary concern of decision makers in market basket analysis. Therefore, HUIM is more suitable than FIM in such scenarios. Furthermore, there are several studies about whether these two dimensions could be integrated to discover more valuable patterns. High-utility frequent itemsets are useful in some cases [12], [13]; but high-utility rare itemset mining (HURIM) [14], [15] also makes sense. Until now, utility mining has been supposed to provide more insight into specific scenarios such as customer segmentation [16], anomaly detection [17], and medical diagnosis [18].

Note that most previously developed HURIM algorithms return a complete set of eligible itemsets that satisfy a user-specified criterion. Under massive databases, the obtained results may still be massive, and not all of them are what the user really needs. Enormous useless or irrelevant results will trouble the user's decisions. For instance, in the traditional shopping basket task, the solution always recommends a wide range of highly profitable products to users. In most cases, however, people go to a retail store because they already know what one or two items they want. This case urges retailers to make more precise recommendations for customers' requirements. Specifically, retailers should sell goods as profitably as possible and meet customers' target goods at the same time. As a result, to represent the interesting patterns in this domain, we define the concept called target high-utility rare itemset (THURI). Moreover, in the field of disease detection [18], targeted mining of high-utility rare itemsets (TMHURI) integrates the doctor's understanding of the disease with the patient's abnormal characteristics (as a target) and then makes the detection process more professional and specialized. According to expert knowledge, in network intrusion detection and tracing, we determine certain behaviors are necessary to achieve the purpose of an attack. TMHURI can be used to enable more efficient localization of anomalies. In the same way, TMHURI can play a more focused role in scenarios in which HURIM has already been applied.

To the best of our knowledge, there has not been any study of the HURIM problem with targeted mining. It is a difficult task to consider these constraints effectively and solve the problem optimally. In summary, how to develop an effective algorithm for TMHURI is research-worthy. In this paper, we present a novel mining task called targeted mining of rare high-utility patterns and propose an algorithm called TaRP.

*Corresponding author.

The major contributions of this paper are listed below.

- In this paper, we study the problem of targeted mining of high-utility rare itemsets for the first time. We propose some key definitions and formulate the problem.
- We propose an algorithm named Targeted mining of Rare high-utility Patterns (TaRP) for solving the defined problem. A data structure called *utility-list* is adopted to hold the heuristic information. This list-based structure can prevent the construction of redundant candidates and effectively reduce search space.
- To improve efficiency, we propose several effective pruning strategies that utilize several properties of utility and support to significantly improve the performance of the targeted mining algorithm.
- We conduct comprehensive experiments on real-world datasets to evaluate TaRP's performance, and the experimental findings indicate that the proposed method has superior performance in terms of effectiveness.

The remainder of this paper is divided into several sections. In Section II, we focus on related work, which is divided into three categories. Section III offers essential preliminaries and formulates the problem. Section IV describes the proposed TaRP algorithm in detail. The experiments and the detailed analysis are shown in Section V. Finally, Section VI presents the conclusion and future work.

II. Related Work

Data mining technologies play a critical role in the big data era. In this section, we provide a brief overview of current research in the subject of itemset mining, including support-based, utility-based, and target-based itemset mining.

A. Support-based itemset mining

Frequent itemset mining (FIM) [3] aims to find itemsets in a database whose occurrence times (support) are no less than a user-defined minimal threshold. In other words, the FIM algorithms mainly consider the support metric. It is clear that if an itemset is frequent, its subsets must also be frequent; if the itemset is infrequent, then all its supersets must be infrequent as well (i.e., the downward closure property [4]). However, traditional FIM algorithms (e.g., Apriori-like) do not perform very well because they iteratively generate candidates and then assess candidates individually. Zaki *et al*. [19] proposed the Eclat algorithm, which adopts an intersection approach to compute the support of itemsets. Han *et al*. [5] proposed the FP-Growth algorithm and a tree structure called the FP-tree. In addition, other previous studies [20], [21], [22] were proposed later. In short, FIM research is a subfield of data mining. However, in some cases, users may be interested in searching for rare itemsets, i.e., itemsets that occur infrequently in databases. This is because rare itemsets deserve special attention from experts in many fields (e.g., outline detection, biology, and medicine). Rare itemset mining has also been widely studied [17], [23], [24]. Another problem is that support-based pattern mining ignores the intrinsic numerical attributes of different items [25], such as weight, quantity, and unit price. To address this issue, the utility mining task was proposed.

B. Utility-based itemset mining

The task of high-utility itemset mining (HUIM) [9], [10], [11] seeks to locate all high-utility itemsets in a database, i.e., the utility values of these itemsets are all greater than or equal to a user-specified minimum utility threshold. HUIM has already been widely used in massive areas and is particularly useful in market basket analysis compared to FIM [26], [27], [28], [29], [30]. The earliest mining process was simply checking the utility of each itemset in the database and then keeping those that satisfied the given condition. This kind of naive approach will result in the search space becoming enormous. Hence, the Two-Phase algorithm [26] utilizes transaction-weighted utilization (abbreviated as *TWU*) upper-bound to reduce the search space. Inspired by Apriori, Two-Phase finds high-utility itemsets with two phases. After that, other two-phase-based algorithms like UP-Growth [27], IHUP [31], and HUP-Growth [32] are proposed.

However, these two-phase algorithms will require massive runtime and memory because of candidate generation, especially for long transactions. Thus, it is important to develop novel data structures that reduce candidate generation steps or the number of candidates. HUI-Miner [28] is the first one-phase HUIM algorithm which can immediately prune useless candidates during the mining process. Based on the utility-list structure, there have been many advanced algorithms [29], [33], [34], [35], [36]. It has been studied that combining utility-oriented mining with support metric is practical and exciting. On the one hand, high-utility frequent itemset mining [13] may provide more accurate combinations for marketers. On the other hand, high-utility rare itemset mining (HURIM) [15] can be used in specific application scenarios, such as outlier detection [17], disease symptom diagnosis [18], and customer segmentation [16]. HURIM task is an active research field, and many algorithms [15], [37] have been proposed to improve the performance of existing algorithms.

C. Target-based itemset mining

All the above-mentioned pattern mining algorithms are designed to discover all the itemsets by giving minimum support or utility thresholds. However, discovering a complete set of patterns from massive data may be unrealistic, and users may struggle to find the results that they require. Therefore, targeted pattern mining [3], [38], [39], as a current challenge, has attracted much attention in recent years. Kubat *et al*. [40] first developed an approach that converts a transaction database into an itemset-tree and then finds target frequent itemsets. Fournier-Viger *et al*. [41] proposed a memory-efficient itemset tree structure to address the poor memory performance of the original itemset-tree on large datasets. For utility-based pattern mining, Miao *et al*. [39] proposed the target pattern tree for mining all the high-utility itemsets that contain the user-given itemsets. As for the sequence data, a PrefixSpan-based approach [42] was developed to mine the target sequences that

meet multiple conditions for market decisions. Zhang *et al.* [43] proposed two novel upper bounds and a vertical structure and utilized a target-chain to improve mining efficiency. Gan *et al.* [12] recently studied the problem that towards target sequential rules.

To our best knowledge, targeted mining of high-utility rare itemset has not yet been studied. To summarize, the existing high-utility rare itemset mining research still aims at mining a complete set of HURIs, which may not be interesting or interactive, as mentioned above. This motivates us to develop an effective and efficient method for discovering high-utility rare patterns in transaction databases in a concise manner.

III. Preliminary and Problem Statement

In this section, we first explain several fundamental definitions before introducing the problem statement of high-utility rare itemset mining with target constraints.

A. Basic preliminaries

In a transaction database $\mathcal{D}$, a finite set of m distinct items is indicated as $I = \{i_1, i_2, \ldots, i_m\}$, where each item corresponds to an external utility $eu(i)$ (e.g., unit profit). $\mathcal{D} = \{T_1, T_2, \ldots, T_n\}$ consists of a set of n transactions. Each transaction T_{tid} consists of a set of items (known as an itemset), and the *tid* refers to the identification of each transaction. Every item i in a transaction T_{tid} is associated with an internal utility $iu(i, T_{tid})$ (e.g., quantity). Table I is taken as an illustrative example in the following content. In this illustrative case, $I = \{a, b, c, d, e, f\}$, and we set the external utility of items to be 5, 3, 2, 4, 8, and 1, respectively.

TABLE I: Example database

T_{tid}	Transaction	Utility
T_1	$\{b:2\}$ $\{c:3\}$	12
T_2	$\{c:1\}$ $\{d:5\}$	22
T_3	$\{a:2\}$ $\{b:5\}$	25
T_4	$\{a:4\}$ $\{c:3\}$ $\{e:2\}$	42
T_5	$\{b:5\}$ $\{d:3\}$ $\{e:1\}$	35
T_6	$\{e:2\}$	16
T_7	$\{a:1\}$ $\{b:2\}$ $\{c:2\}$ $\{e:5\}$	55
T_8	$\{d:1\}$ $\{e:2\}$	20
T_9	$\{c:1\}$ $\{f:2\}$	4
T_{10}	$\{a:1\}$ $\{b:1\}$ $\{c:1\}$	10

Definition 1: (**Utility calculation**) The utility of an item i in the transaction T_{tid} is defined as $u(i, T_{tid}) = iu(i, T_{tid}) \times eu(i)$. An itemset X is a k-itemset when it contains k different items. The utility of the itemset X in transaction T_{tid} is denoted as $u(X, T_{tid}) = \sum_{i \in X} u(i, T_{tid})$. The utility of a transaction T_{tid} is then denoted as $u(T_{tid}) = \sum_{i \in T_{tid}} u(i, T_{tid})$. The total utility of X in $\mathcal{D}$ can be denoted as $u(X) = \sum_{X \subseteq T_{tid} \wedge T_{tid} \in \mathcal{D}} u(X, T_{tid})$.

Considering the example database in Table I, the utility value of $\{a\}$ in T_3 is $u(a, T_3) = 2 \times 5 = 10$. The utility of itemset $\{a, b\}$ in T_3 is $u(\{a, b\}, T_3) = u(a, T_3) + u(b, T_3) = 2 \times 5 + 5 \times 3 = 25$. Since $T_3 = \{a, b\}$ and $u(T_3) = 25$, then $u(\{a, b\}) = u(\{a, b\}, T_3) + u(\{a, b\}, T_7) + u(\{a, b\}, T_{10}) = u(a, T_3) + u(b, T_3)) + u(a, T_7) + u(b, T_7)) + u(a, T_{10}) + u(b, T_{10}) = 44$.

Definition 2: (**Transaction-weighted utilization**) The transaction-weighted utilization (*TWU*) of an itemset X is the total of all transaction utilities of transactions that contain X: $TWU(X) = \sum_{X \subseteq T_{tid} \wedge T_{tid} \in \mathcal{D}} u(T_{tid})$ [26]. We refer to X as a potential high-utility itemset if $TWU(X)$ is greater than or equal to *minutil*. Table II lists the *TWU* values of all items.

Property 1: (**Anti-monotonicity**) The *TWU* of an itemset X is always no less than its total utility in $\mathcal{D}$, i.e., $TWU(X) \geq u(X)$. Considering X and Y to be two distinct itemsets, consequently, $TWU(X) \geq TWU(Y)$ if $X \subset Y$. The proof detail was provided in [26].

TABLE II: Transaction-weighted utility

Item	a	b	c	d	e	f
TWU	132	137	133	77	168	4

Definition 3: (**Support**) For an itemset X, its support is denoted as $sup(X)$ and defined as $sup(X) = |T_{tid} \mid X \subseteq T_{tid} \wedge T_{tid} \in \mathcal{D}|$. It indicates how many transactions in the database $\mathcal{D}$ contain X.

For example, in Table I, the transactions T_3, T_7, and T_{10} all contain the itemset $\{a, b\}$. As a result, $sup(\{a, b\}) = 3$.

Definition 4: (**High-utility rare itemset**) For an itemset X, if $u(X)$ is no less than a user-specified minimum utility threshold *minutil*, then X is a high-utility itemset. X is a rare itemset if $minsup \leq sup(X) \leq maxsup$. Here, *minsup* and *maxsup* are the minimum and maximum support thresholds given by user. X is called a high-utility rare itemset (HURI) if $u(X) \geq minutil$, $minsup \leq sup(X) \leq maxsup$.

Definition 5: (**Target high-utility rare itemset**) The target itemset (*TI*) is a user-specified set of items in which the user is interested, where $|TI| \geq 1$ [39]. Given a high-utility rare itemset X, if $TI \subseteq X$, we suppose X is a target high-utility rare itemset (THURI). Thus, it satisfies target constraint.

B. Problem statement

In a transaction database $\mathcal{D}$, given a set of target items *TI*, a minimum utility threshold *minutil*, and two support thresholds (*minsup* and *maxsup*). The goal of targeted mining of high-utility rare itemsets (abbreviated as TMHURI) is to find all the high-utility rare itemsets containing *TI* among the given data, with a set of parameter settings. For any X in THURIs, it has $TI \in X$, $u(X) \geq minutil$, and $minsup \leq sup(X) < maxsup$.

In Table I, assume that given a group of parameters: $TI = \{a, e\}$, $minutil = 55$, $minsup = 1$, and $maxsup = 3$. We then can discover the existence of three THURIs: $\{a, b, c, e\}$(u: 55, sup: 1), $\{a, e\}$(u: 81, sup: 2), and $\{a, c, e\}$(u: 91, sup: 2).

IV. The TaRP Algorithm

In this section, we introduce a detailed presentation of the TaRP algorithm. In subsection IV-A, we first introduce the utility-list structure for storing the heuristic information during the mining process. Then, we illustrate the target set-numeration tree and matching mechanism in subsection IV-B. Subsection IV-C presents several efficient pruning strategies. In subsection IV-D, we describe the pseudocode of the proposed algorithm.

A. Utility-list structure

Definition 6: (**Remaining utility** [28]) Given an itemset X and a transaction T with $X \subseteq T$, the set of all items appearing after X in T is denoted as T/X. In addition, for an itemset X, the sum of the utility values of all items in T_{tid}/X is defined as X's remaining utility in transaction T_{tid}, denoted as *rutil*(X, T_{tid}). Mathematically, *rutil*(X, T_{tid}) = $\sum_{i \in (T_{tid}/X)} u(i, T_{tid})$.

Take Table I as an illustrative example. For the itemset $\{a, c\}$ in transaction T_7, the remaining items of $\{a, c\}$ in T_7 is $T_7/\{a, c\}$ = $\{e\}$, and the remaining utility of $\{a, c\}$ in T_7 is $u(\{e\}, T_7)$ = 40. The utility-list was originally proposed in HUI-Miner [28] to store the utility information of potential high-utility itemsets. As shown in Fig. 1, a utility-list is a set of tuples. Each tuple consists of three elements: <*tid*, *iutil*, *rutil*> [28]. The *tid* identifies a transaction containing the given itemset X; the *iutil* element is the utility of X in T_{tid} (i.e., $u(X, T_{tid})$); and the *rutil* element is an abbreviation of the remaining utility, defined as $\sum_{i \in (T_{tid}/X)} u(i, T_{tid})$. In Fig. 1, we assume the first element column *tids* is a set of *tid*. Similarly, *iutils* and *rutils* are the sets of *iutil* and *rutil*, respectively. Using this structure, we can effectively obtain heuristic information about an itemset.

Given an itemset X and its utility-list *ULOfX*, we can calculate its support value by examining the number of elements in *ULOfX*, or directly compute the size of *tids*. The total utility value of X in $\mathcal{D}$ is the summary utility value of *iutils*. In addition, the total remaining utility value of X in $\mathcal{D}$ is used to determine whether X is promising. We can build the utility-list of (k+1)-itemset by intersecting two utility-lists of different k-itemsets, where $k \geq 1$.

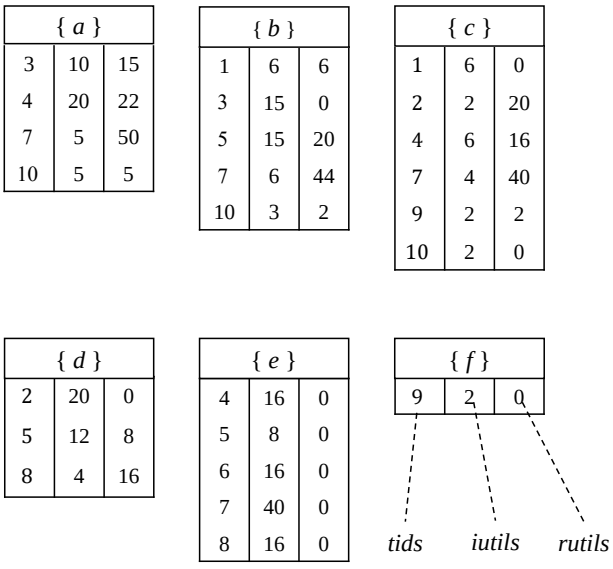

Fig. 1: All utility-lists of 1-itemsets

For illustration, based on Fig. 1, the utility-list of $\{a\}$ is {<3, 10, 15>, <4, 20, 22>, <7, 5, 50>, <10, 5, 5>}, the utility-list of $\{b\}$ is {<1, 6, 6>, <3, 15, 0>, <5, 15, 20>, <7, 6, 44>, <10, 3, 2>}. To obtain the utility-list of $\{a, b\}$, we don't need to scan the database again; we just need to intersect the utility-lists of $\{a\}$ and $\{b\}$. After a calculation based on the intersection rule for utility-lists, the utility-list of $\{a, b\}$ is {<3, 25, 0>, <7, 11, 44>, <10, 8, 2>}. The numbers of tuples in the utility-list of $\{a\}$ and $\{b\}$ are 4 and 5, respectively, which represent the support value of $\{a\}$ and $\{b\}$; and $sup(\{a, b\})$ is 4, the smaller of these two numbers.

B. Search space and matching mechanism

With the *TI* setting, the search space of our proposed algorithm can be seen as a target set-enumeration tree. In set-enumeration tree [44], the nodes of the tree are regarded as representations of itemsets. In this paper, the *total order* of items always takes the *TWU*-ascending order [33]. For example, if the user-specified threshold *minutil* is 40 since *TWU*(f) < *minutil*, neither f nor its supersets will be HURI. Thus, the final total order is $d \prec a \prec c \prec b \prec e$. Based on Table I, the whole targeted search tree is shown in Fig. 2.

Definition 7: (**Extension**) Given a target set-enumeration tree, for an itemset represented by a node, each of its descendants is called the extension of the itemset. For an itemset containing k items, its extension containing (k+i) items is called an i-extension of this itemset.

Definition 8: (**IMatch**) Considering an itemset X and a target itemset *TI*, if *TI* is a subset of X (i.e., *TI* $\subseteq X$), then *TI* *Match* X or X can be *Matched* by *TI*. Given $X = \{i_1, i_2, \ldots, i_m\}$ and *TI* = $\{t_1, t_2, \ldots, t_n\}$ where $n \leq m$, if an item t_{index} is identical to an item i_i ($1 \leq i \leq m$ and $1 \leq index \leq n$), then we suppose *TI IMatch* (item match) X or X can be *IMatch* by *TI*. The *index* here indicates the current item to be matched in *TI* and will be updated during the matching process. In addition, for each item in *TI* (t_1 to t_n), if an item i_i in X has $t_{index} = i_i$, then *TI Match* X[1].

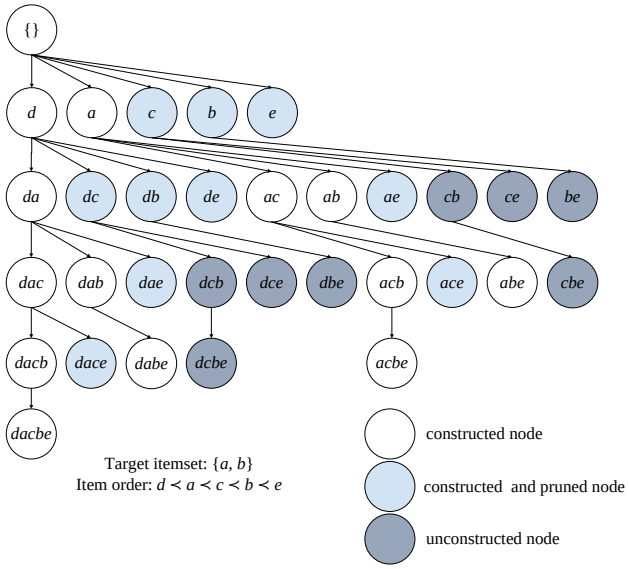

Fig. 2: A target set-enumeration tree

[1]It should be pointed out that *IMatch* represents *TI* $\cap$ $X \neq \emptyset$, but *Match* means *TI* $\subseteq X$.

C. Effective pruning strategies

Strategy 1: (**TWU-Prune**) Let X be an itemset. According to Property 1, if $TWU(X) < minutil$, then neither X nor any of its supersets are high-utility itemsets.

Strategy 2: (**UT-Prune**) The super-itemsets of *TI* are only generated from transactions that contain *TI*, so removing transactions that do not contain *TI* (unpromising transactions, abbreviated as UT) in advance can significantly reduce the memory consumption for loading $\mathcal{D}$.

Strategy 3: (**IMatch-Prune**) When *TI IMatch* current itemset X in the search space (i.e., target set-enumeration tree), the search for the tree(s) rooted at sibling(s) of X can be terminated.

Proof: We assume that each item $i \in I$ and *TI* is sorted in the *TWU*-ascending order, and the search space is based on the same order. Hence, let P be an itemset node (except leaf nodes) and *Px* be a *1*-extension of *P*, we suppose that *Px* is matched by *TI* as the current searched node, where $x \in TI$. For any sibling node (*Py*) of *Px*, since $x \prec y$ and $x \neq y$, the subtree of *Py* (the root node is *Py*) does not contain X, and we can conclude that *TI* does not *Match* all nodes of *Py* subtree. In other words, a depth-first search for trees rooted at the siblings of *Px* is useless, because none of the nodes in these subtrees contain the entire *TI*. ■

To demonstrate, consider the target set-enumeration tree in Fig. 2 with an expansion order of $d \prec a \prec c \prec b \prec e$. The goal is to search for the itemsets that contain $TI = \{a, b\}$ using *IMatch-Prune*. For a depth-first search of a tree that rooted with *1*-itemset $\{d\}$, the first extension is $\{d, a\}$. We can find that *TI IMatch* $\{d, a\}$ at the time, because the current item to be *IMatch* in *TI* is a, that is the same extended item in $\{d, a\}$. Therefore, the sibling nodes of the node $\{d, a\}$ (i.e., $\{d, c\}$, $\{d, b\}$ and $\{d, e\}$), will not contain a. Similarly, if the nodes that inherit these nodes do not contain a, none of them will contain *TI*. These nodes can be pruned by the *IMatch-Prune* strategy. The proposed algorithm adopts utility-lists primarily for holding the utility and support information of itemsets. There are several effective pruning strategies capable of improving performance.

Strategy 4: (**RU-Prune** [28]) For an itemset X, if the sum of *iutils* and *rutils* of X is less than the given *minutil* threshold, all supersets of X (i.e., all extensions of X) are not high-utility itemsets, so X and its supersets can be pruned.

Strategy 5: (**LA-Prune** [45]) Given two itemset *Px* and *Py* with the common prefix *P*, if $\sum_{T_i \in \mathcal{D} \wedge Px \subseteq T_i} (iutil(Px, T_i) + rutil(Px, T_i)) - \sum_{T_j \in \mathcal{D} \wedge Px \subseteq T_j \wedge Py \not\subseteq T_j} (iutil(Px, T_j) + rutil(Px, T_j)) < minutil$, then itemset *Pxy* and its extensions are not HUIs.

Strategy 6: (**Sup-Prune**) If the support of an itemset X is less than *minsup*, X and its supersets can be directly pruned.

Strategy 7: (**RSup-Prune**) Given two itemsets *Px* and *Py*, if $sup(Px) - |T_{tid}|Px \subseteq T_{tid} \wedge Py \not\subseteq T_{tid} \wedge T_{tid} \in \mathcal{D}| < minsup$, then the itemset *Pxy* and all of its extensions are certainly not rare itemsets.

Proof: Mathematically, let Δ be $sup(Px) - |T_{tid}|Px \subseteq T_{tid} \wedge Py \not\subseteq T_{tid} \wedge T_{tid} \in \mathcal{D}|$, expanding the $sup(Px)$ of the expression:

$$
\begin{aligned}
\because sup(Px) &= |T_{tid}|Px \subseteq T_{tid} \wedge T_{tid} \in \mathcal{D}| \\
&= |T_{tid^*}|Px \subseteq T_{tid^*} \wedge Py \subseteq T_{tid^*} \wedge T_{tid^*} \in \mathcal{D}| \\
&\quad + |T_{tid}|Px \subseteq T_{tid} \wedge Py \not\subseteq T_{tid} \wedge T_{tid} \in \mathcal{D}| \\
\therefore \Delta &= |T_{tid}|Px \subseteq T_{tid} \wedge Py \subseteq T_{tid} \wedge T_{tid} \in \mathcal{D}| \\
&= sup(Pxy), \text{ by Def. 3.}
\end{aligned}
$$

Therefore, let χ be any of the extensions of *Pxy*, if $sup(Pxy) < minsup$, then $sup(\chi) < minsup$ according to the anti-monotonicity of support measure. ■

Inspired by the EUCS structure in FHM [29], we propose a novel Estimated Utility-Support Co-occurrence Structure (abbreviated as EUSCS) for storing heuristic information of k-itemsets ($k \geq 2$). Then, we propose the Estimated Utility-Support Co-occurrence Pruning strategy.

Definition 9: Estimated Utility-Support Co-occurrence Structure (EUSCS) is defined as a set of quadruple of the form $(x, y, u, s) \in I^* \times I^* \times \mathbb{R} \times \mathbb{N}$, where $TWU(\{x, y\}) = u$ and $sup(\{x, y\}) = s$. The entire structure can be viewed as the matrix-like structure, as shown in Fig. 3. Note that only tuples with $s \neq 0$ will be saved in the implementation.

Property 2: In the process of merging itemsets *Px* and *Py* to get a new itemset *Pxy* (*P* is the common prefix), if there exists a tuple (x, y, u, s) in EUSCS that $u < minutil$, then *Pxy* and all its supersets are not high-utility itemsets based on Property 1. Analogously, if there exists a tuple (x, y, u, s) in EUSCS with $s < minsup$, then *Pxy* and all its supersets are not rare itemsets based on Strategy 6.

Strategy 8: (**EUSC-Prune**) If there is no tuple (x, y, u, s) in EUSCS with $u \geq minutil$ and $s \geq minsup$, then the itemset *Pxy* and all its supersets are not HURIs based on Property 2.

TWU / Sup: Item \ Item	*d*		*a*		*c*		*b*	
a	0	0						
c	22	1	107	3				
b	35	1	90	3	77	3		
e	55	2	97	2	97	2	90	2

Fig. 3: Estimated utility-support co-occurrence structure

The join operation that obtains the *k*-itemset based on the list structure is resource-intensive. *EUSC-Prune* can effectively reduce this construction process to improve performance (V-D). For example, with *minutil* = 30 and *minsup* = 2, we can know from EUSCS that the value of $TWU(\{d, c\})$ is 22 (less than *minutil*). Although $TWU(\{d, b\})$ is 35 (greater than *minutil*), its support is 1 and less than *minsup*, so neither of these two itemsets nor their supersets are HURIs.

D. The proposed algorithm

Herein, we described the proposed algorithm in detail. The pseudocode of the main procedure is presented in Algorithm

Algorithm 1: The TaRP algorithm

Input: $\mathcal{D}$: a transaction database, *TI*: target pattern, *minutil*: the minimum utility threshold, *minsup*: the minimum support threshold, *maxsup*: the maximum support threshold.

Output: all the targeted high-utility rare itemsets.

1 scan $\mathcal{D}$ to filter out the redundant transactions, then calculate the $sup(i)$ and $TWU(i)$ for each item $i \in I$;
2 $I^* \leftarrow \{i | i \in I \wedge TWU(i) \geq minutil \wedge sup(i) \geq minsup\}$;
3 remove item $i \notin I^*$ from each transaction to obtain revised database $\mathcal{D}^*$;
4 **for** each t_i in *TI* **do**
5 **if** $TWU(t_i) < minutil$ **or** $sup(t_i) < minsup$ **then**
6 **return** *null*;
7 **end**
8 **end**
9 sort $i \in I^*$ by the *TWU* ascending order $\prec$;
10 sort *TI* by the *TWU* ascending order $\prec$;
11 scan $\mathcal{D}^*$ to construct the utility-list (UL) of each promising item and then build the *EUSCS*;
12 call **Search**($\emptyset$, *ULs*, *minutil*, *minsup*, *maxsup*, 0);

Algorithm 2: The Search procedure

Input: *ULOfP*: utility-list of prefix itemset *P*, *ULs*: a set of utility-lists of *1*-extensions of itemset *P*, *minutil*: the minimum utility threshold, *minsup*: the minimum support threshold, *maxsup*: the maximum support threshold, *index*: points to the current item in *TI* that needs to be matched.

Output: a set of the target high-utility rare itemsets (THURIs).

1 initialize *IMatch* $\leftarrow$ false;
2 **for** each utility-list of itemset *Px* in *ULs* **do**
3 initialize *currentIndex* $\leftarrow$ *index*;
4 **if** *IMatch* == *True* **then**
5 break;
6 **end**
7 **if** *currentIndex* $<$ $|TI|$ **and** *x.item* == *TI*[*index*] **then**
8 *currentIndex*++;
9 *IMatch* $\leftarrow$ *True*;
10 **end**
11 **if** *Px.sumIutils* $\geq$ *minutil* **and** *currentIndex* $\geq$ $|TI|$ **then**
12 **if** *sup(Px)* $\geq$ *minsup* **and** *sup(Px)* $<$ *maxsup* **then**
13 output *Px*;
14 **end**
15 **end**
16 **if** *Px.sumIutils* + *Px.sumRutils* $\geq$ *minutil* **and** *sup*(*Px*) $\geq$ *minsup* **then**
17 *newULs* $\leftarrow$ *null*;
18 **for** each utility-list *Py* in ULs that $x \prec y$ **do**
19 **if** $\exists(x, y, u, s) \in$ *EUSCS* that $u \geq$ *minutil* **and** $s \geq$ *minsup* **then**
20 *newULs* $\leftarrow$ *newULs* $\cup$ **construct**(*ULOfP*, *Px*, *Py*);
21 **end**
22 **end**
23 **end**
24 call **Search**(*ULs*, *newULs*, *minutil*, *minsup*, *maxsup*, *currentIndex*);
25 **end**

1. It takes a transaction database, a target itemset, a minimum utility threshold, a minimum support threshold, and a maximum support threshold as input parameters. The end result is a complete set of target-constrained rare itemsets with high utility. In line 1, TaRP performs a first scan of the original database $\mathcal{D}$ and removes the transactions that do not contain *TI* (*UT-Prune*, Strategy 2). After the database is filtered, the *TWU* and support of each item are computed for further use. A revised database $\mathcal{D}^*$ is then obtained by removing all unpromising items using *TWU-Prune* (Strategy 1) and *Sup-Prune* (Strategy 6) (lines 2–3). If *TWU* or support of any item in *TI* is less than the corresponding thresholds, there is no itemset that *Match TI* in $\mathcal{D}$ (lines 4–8). Otherwise, TaRP sorts I^* and *TI* by the *TWU*-ascending order $\prec$, thereby reducing the number of constructed utility-lists significantly (lines 9–10). Next, the utility-lists of each retained *1*-itemset are built after the second database scan, and EUSCS is also created in the process (line 11). Finally, a recursive method is called to mine all THURIs in a depth-first search (line 12).

Algorithm 2 illustrates the recursive mining process in detail. The overall process is a determination of whether the extensions of the prefix itemset *P* are the targeted itemsets or not (i.e., itemsets that satisfy the constraints). At first, the procedure initializes *IMatch* as *false*, meaning that the *IMatch* event does not occur at the beginning (line 1). Then, the method uses a for-loop to iterate over the *1*-extensions of *P* (that is, the children of node *P* in the set-enumeration tree). If *TI IMatch* itemset *Px* during the iteration, the recursive search for the follow-up itemsets is stopped according to *IMatch-Prune* (lines 2–10). If the search process reveals that *Px* contains *TI* and satisfies the utility and support conditions, then a new THURI is discovered (lines 11–14). If there are potential THURIs in the extensions of *Px*, then the procedure adopts the strategy *EUSC-Prune* (Strategy 8) before constructing utility-lists of these extensions (line 16). If there exists a tuple (*x, y, u, s*) in EUSCS where *u* and *s* are no less than *minutil* and *minsup*, respectively, the method will call the *Construct* procedure (cf. Algorithm 3) to build the utility-list of *Pxy* (lines 19–21). Finally, after all the potential utility-list of extensions of *Px* are collected, the procedure recursively calls itself to explore all high-level THURIs (line 24).

Algorithm 3 describes the process of building a utility-list for itemset *Pxy*. It takes the utility-list of prefix itemset *P*, the utility-lists of *Px* and *Py* as input parameters. While the algorithm mines 1-itemsets, *P* and *ULOfP* are $\emptyset$ and

Algorithm 3: The Construct procedure

Input: *ULOfP*, *ULOfPx*, *ULOfPy*: the utility-list of itemset *P*, *Px* and *Py*.
Output: *ULOfPxy*: the utility-list of itemset *Pxy*.

```
1  ULOfPxy ← null;
2  Utility ← SUM(Px.Iutils) + SUM(Px.Rutils);
3  Sup ← |ULOfPx|;
4  for each element ex in ULOfPx do
5      if ∃ ey ∈ ULOfPy and ex.tid == ey.tid then
6          if ULOfP ≠ null then
7              search such element e that e.tid == ex.tid;
               new exy ← <ey.tid, ex.iutil + ey.iutil −
               e.iutil, ey.iutil>;
8          else
9              new exy ← <ey.tid, ex.iutil + ey.iutil,
               ey.iutil>;
10         end
11         ULOfPxy ← ULOfPxy ∪ exy;
12     else
13         Utility ← Utility − ex.iutil − ex.rutil;
14         Sup−−;
15         if Utility < minutil or Sup < minsup then
16             return null;
17         end
18     end
19 end
20 return ULOfPxy
```

null, respectively. The construction process is the same as in HUI-Miner [28]. In addition, itemsets that don't satisfy the constraints of utility and support thresholds will be pruned because of the application of *LA-Prune* (Strategy 5) and *RSup-Prune* (Strategy 7) in the procedure (lines 13–17). Note that the construction process will be terminated early if the subtracted value is less than *minsup*, during the decrement-by-decrement calculation of $sup(Pxy)$.

V. Experiments

In this section, we perform several experiments on real-world datasets to assess the effectiveness and efficiency of our proposed TaRP algorithm. To our best knowledge, TaRP is the first algorithm for targeted mining of HURIs. We also utilize a post-processing technique to evaluate the correctness of the experimental results.

A. Post-processing technique

The post-processing technique can be used to verify the correctness of the patterns found by the algorithm. We used it to discover the complete set of items, and the modified algorithm can be seen as a benchmark for validation. In brief, this technique discovers HURIs that satisfy both support and utility constraints. It compares the mined itemset with the targeted itemset *TI*, if an itemset contains *TI*, then this itemset will be saved. Otherwise, it will be filtered.

B. Experimental setup

The performance experiments were conducted on three real-world datasets. Copies of the datasets are accessible via download through an open-source SPMF website[2]. Table III displays the characteristics of the selected datasets. We have listed the characteristics of the chosen dataset separately: the first column is the name of the dataset; $|\mathcal{D}|$ is the number of transactions contained in $\mathcal{D}$; $|I|$ is the number of unique items in $\mathcal{D}$; AvgLen is the average length of the itemsets in $\mathcal{D}$; and density refers to the value obtained by dividing AvgLen by $|I|$. We selected and fixed the range of support in the experiment based on the size and density of the chosen datasets as a way to ensure the reliability of the experiment. The support rate $\times$ $|\mathcal{D}|$ is used to set the support values of each dataset, and the details are shown in Table IV. Consider the *minrate* and *maxrate* columns in Table IV. For instance, "0.1% (15)" means that the support rate is 0.1% and the corresponding support value is 15; "0.3% (45)" means that the support rate is 0.3% and the corresponding support value is 45.

TABLE III: Features of the datasets

Dataset	$\|\mathcal{D}\|$	$\|I\|$	AvgLen	Density
ecommerce	14,975	3,468	11.71	0.34%
mushroom	8,416	119	23	19.33%
connect	67,557	129	43	33.33%

TABLE IV: Support setting on each dataset

Dataset	$\|\mathcal{D}\|$	minrate	maxrate
ecommerce	14,975	0.1% (15)	0.3% (45)
mushroom	8,416	1% (84)	3% (254)
connect	67,557	0.3% (203)	0.8% (540)

The experiments were carried out on a computer equipped with an Intel(R) Core(TM) i5-8300H CPU running at 2.30 GHz, the 64-bit Windows 10 operating system, and 16 GB of RAM. All compared algorithms were implemented in the Java language. For reproducibility, we make the source code publicly available on GitHub[3].

In the following experiments, we designed two variants of TaRP based on different pruning strategies. TaRP$_{V1}$ utilizes *TWU-Prune*, *UT-Prune*, and the post-processing technique. TaRP$_{V2}$ utilizes all of the aforementioned prune strategies. To confirm the correctness of the mining results of our proposed method, we chose the HURI-Miner algorithm [15] which adds the port-processing technique, as a strong baseline for comparison. We also ensured that the support definition of HURI-Miner is the same as that of our algorithm to ensure fairness in the experimental comparison. We renamed the modified algorithm as HURI-Miner*. If the tested algorithm runs over 3,600 seconds, we suppose it cannot complete the given task within a valid time frame and use "/" to represent its results in the tables.

[2]http://www.philippe-fournier-viger.com/spmf/index.php
[3]https://github.com/DSI-Lab1/TargetUM

TABLE V: Number of returned itemsets under various *minutil*

Dataset	Algorithm	$minutil_1$	$minutil_2$	$minutil_3$	$minutil_4$	$minutil_5$	$minutil_6$	$minutil_7$
ecommerce	HURI-Miner*	6	6	6	6	5	4	4
	$TaRP_{V1}$	6	6	6	6	5	4	4
	$TaRP_{V2}$	6	6	6	6	5	4	4
mushroom	HURI-Miner*	138,169	90,962	55,085	30,134	14,715	6,272	2,246
	$TaRP_{V1}$	138,169	90,962	55,085	30,134	14,715	6,272	2,246
	$TaRP_{V2}$	138,169	90,962	55,085	30,134	14,715	6,272	2,246

C. Itemset analysis

In contrast to HURIM, which has received extensive research, TaRP considers the user's subjective (i.e., the target itemsets). We use HURI-Miner* as a baseline for comparison. As for the *TIs* of the different datasets in the experiments, we set the ecommerce *TI* = {21754, 21755}, mushroom *TI* = {28, 114}, and connect *TI* = {123, 125}, respectively, and the *TIs* in the performance analysis below are the same settings. Note that the various *minutil* settings in Table V and Table VI are consistent in the experiments with the various *minutil* for the efficiency analysis.

As shown in Table V, we can observe that the number of THURIs found by $TaRP_{V1}$ is always the same as that discovered by HURI-Miner* and $TaRP_{V2}$. This demonstrates the correctness of the results mined by the proposed TaRP algorithm. Besides, we can conclude that the pruning strategies enable the algorithm to improve its efficiency while maintaining the same results.

D. Efficiency analysis

In this subsection, we performed several experiments to evaluate the performance of the compared algorithms. In each dataset, we can observe the performance of the algorithm while continuously increasing the *minutil* threshold and fixing the other parameters. Runtime and memory consumption are important indicators to determine whether the performance of an algorithm is acceptable. The final experimental results we obtained for both of these at their specific parameter settings are shown in Fig. 4 and Fig. 5.

Overall, $TaRP_{V2}$ with all pruning strategies achieves the fastest running time on all tested datasets. On the sparse dataset *ecommerce*, the runtime performance of HURI-Miner* is better than that of $TaRP_{V1}$ but worse than that of $TaRP_{V2}$. This indicates that, on the one hand, $TaRP_{V1}$'s strategy of filtering irrelevant transactions on sparse datasets is insufficient to outperform the baseline; on the other hand, $TaRP_{V2}$'s additional pruning strategies are very efficient. On the dense dataset (e.g., *mushroom*), HURI-Miner* uses significantly more runtime than the two variants of TaRP under specific experimental parameter settings. Specifically, when *minutil* equals 40k[4], HURI-Miner* takes more than an order of

[4]Unless we wish to stress the difference, "k" always represents 1,000 in the following content.

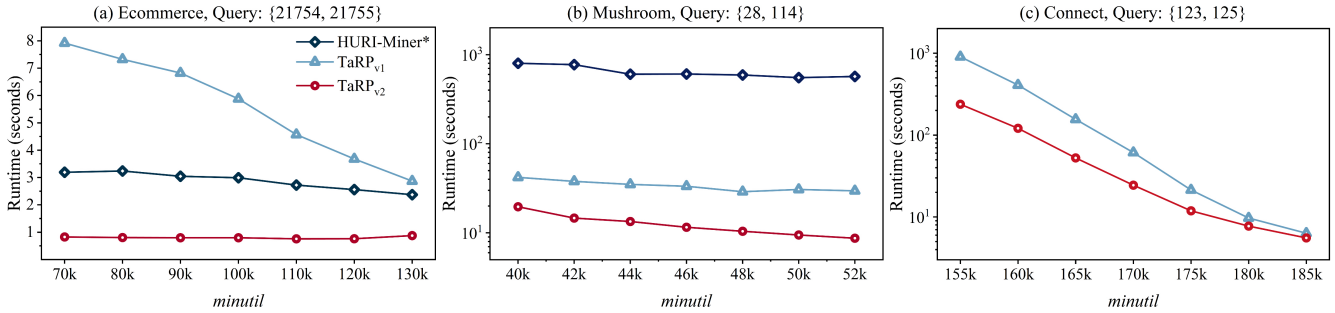

Fig. 4: Runtime usage under various *minutil*

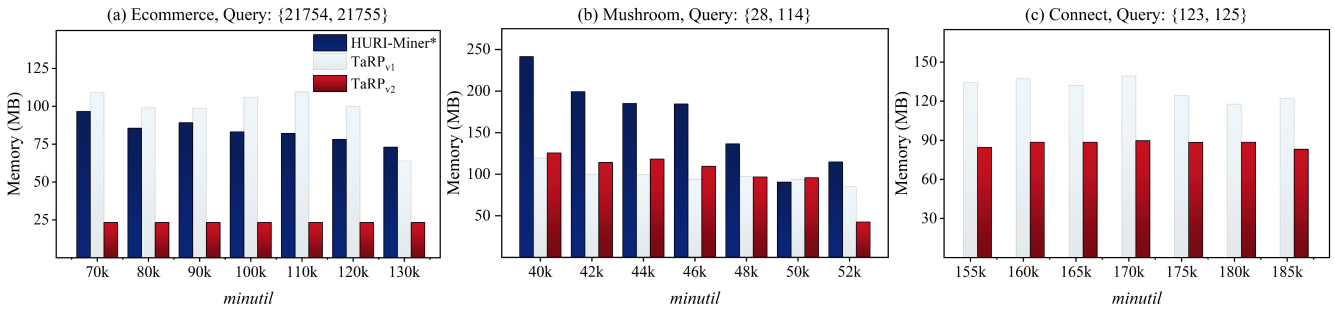

Fig. 5: Memory usage under various *minutil*

TABLE VI: Number of intersection under various *minutil*

Dataset	Algorithm	$minutil_1$	$minutil_2$	$minutil_3$	$minutil_4$	$minutil_5$	$minutil_6$	$minutil_7$
ecommerce	HURI-Miner*	18,269	18,269	18,265	18,259	18,253	18,240	18,222
	TaRP$_{V1}$	163,812,808	130,339,194	117,835,528	102,057,916	83,147,043	63,020,438	43,973,329
	TaRP$_{V2}$	15	15	15	15	15	15	15
mushroom	HURI-Miner*	18,310,644	16,285,281	14,498,557	12,931,279	11,584,445	10,452,792	9,474,723
	TaRP$_{V1}$	1,400,403	1,172,030	977,513	823,175	705,507	616,400	547,405
	TaRP$_{V2}$	462,886	391,713	325,449	268,247	222,218	188,272	163,163
connect	HURI-Miner*	/	/	/	/	/	/	/
	TaRP$_{V1}$	15,853,637	6,985,313	2,673,381	959,880	271,862	66,864	12,820
	TaRP$_{V2}$	5,335,989	2,455,072	994,782	377,296	116,538	31,169	6,017

magnitude longer in runtime than TaRP$_{V1}$, while TaRP$_{V2}$ is much faster than TaRP$_{V1}$ as threshold rises. As for the larger dataset (e.g., *connect*), HURI-Miner* is unable to complete the discovery task in the experiment. Both variants of TaRP can mine complete results that satisfy the given conditions in a reasonable time, and TaRP$_{V2}$ is almost always faster than TaRP$_{V1}$. Overall, the change in runtime consumption for TaRP$_{V2}$ is stable.

We also assess the efficiency of TaRP for mining in relation to memory consumption. On the *ecommerce* dataset, we can see that both TaRP$_{V1}$ and baseline consume far more memory than TaRP$_{V2}$ at different experimental *minutil* settings. This is consistent with the findings from the previous runtime analysis. It is clear that the change in memory consumption for TaRP$_{V2}$ is stable. On the *mushroom* dataset, HURI-Miner* almost always consumes more memory than the TaRP algorithm, especially when *minutil* is relatively small: *minutil* equals 40k, which uses about twice as much memory as TaRP. After comparing the two variants of TaRP, we find that TaRP$_{V2}$ consumes slightly more memory than TaRP$_{V1}$ in some cases. This is because storing EUSCS consumes some memory, but the difference is not significant. In some cases, such as *minutil* equals to 52k, the *EUSC-Prune* strategy gains the advantage of both runtime and memory consumption. On the large dataset *connect*, the baseline cannot complete the task. The memory cost calculated by TaRP maintains a stable range. In addition, at the same parameter settings, TaRP$_{V1}$ consumes roughly 1.5 times the memory of TaRP$_{V2}$. In general, empirical evidence shows that the proposed algorithm performs well in terms of memory consumption.

To further evaluate the efficiency of the pruning strategies, we also experimented with the candidates generated during the operation of the algorithm, as shown in Table VI. In a utility-list-based algorithm, the process of generating candidate utility-lists has a significant impact on performance. For instance, on the *ecommerce* dataset, TaRP$_{V1}$ produces massive candidates, because it only adopts few strategies compared to TaRP$_{V2}$. In contrast to the baseline algorithm, TaRP$_{V2}$ can significantly reduce candidate generation in the experimental setup. This also confirms that the large number of candidates generated is the main cause of the high running time and memory consumption mentioned above. On the *mushroom*, a dense dataset, we can see that since TaRP$_{V1}$ utilizes the strategy *UT-Prune*, the number of candidates is reduced by more than an order of magnitude compared to that of the baseline. Therefore, the comparison test shows that *UT-Prune* is more effective on the dense dataset. With the benefit of the strategies adopted in TaRP$_{V2}$, the number of candidates generated by TaRP$_{V2}$ is reduced even further, to essentially one-third of the number generated by TaRP$_{V1}$. Obviously, the performance advantage of TaRP is also validated on the *connect* dataset. The baseline HURI-Miner* is unable to complete the mining process on this larger and denser dataset, while two variants of TaRP can discover interesting results within acceptable timescales.

VI. Conclusion

Current algorithms for mining HURI are only able to return a complete set of HURIs that satisfy the utility and support constraints on transaction databases, but they are not all interesting. In this paper, we define a new task, namely targeted mining of high-utility rare itemsets (TMHURI), and propose an algorithm for effectively discovering targeted high-utility rare itemsets. TMHURI is interactive and specific in pattern discovery because it incorporates the subjective goals of users. It is better able to provide users with abnormal but vitally interesting patterns in real-world applications. To reduce the algorithm's execution time, we propose several effective pruning strategies to overcome the corresponding constraints, and the analysis of experimental results shows that these strategies achieve our objective.

In the future, we believe that targeted pattern mining is a realistic research direction for data mining, especially for concise pattern search. In the field of pattern mining, especially targeted utility mining, it is worth practicing how to develop more powerful pruning strategies and algorithms that work for different types of rich data (e.g., dynamic data, uncertain data, streaming data, and sequence data).

Acknowledgment

This research was supported in part by the National Natural Science Foundation of China (Nos. 61902079, 62002136, and 62272196), Natural Science Foundation of Guangdong Province (No. 2022A1515011861), Guangzhou Basic and Applied Basic Research Foundation (Nos. 202102020277 and 202102020928), Fundamental Research Funds for the Central Universities of Jinan University (No. 21622416), and the Young Scholar Program of Pazhou Lab (No. PZL2021KF0023). Dr. Wensheng Gan is the corresponding author of this paper.

REFERENCES

[1] W. Gan, J. C. W. Lin, H. C. Chao, and J. Zhan, "Data mining in distributed environment: a survey," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 7, no. 6, p. e1216, 2017.

[2] W. Gan, J. C. W. Lin, P. Fournier-Viger, H. C. Chao, and P. S. Yu, "A survey of parallel sequential pattern mining," *ACM Transactions on Knowledge Discovery from Data*, vol. 13, no. 3, pp. 1–34, 2019.

[3] P. Fournier Viger, W. Gan, Y. Wu, M. Nouioua, W. Song, T. Truong, and H. Duong, "Pattern mining: Current challenges and opportunities," in *The International Conference on Database Systems for Advanced Applications*. Springer, 2022, pp. 34–49.

[4] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules," in *The 20th International Conference on Very Large Data Bases*, vol. 1215. Citeseer, 1994, pp. 487–499.

[5] J. Han, J. Pei, and Y. Yin, "Mining frequent patterns without candidate generation," *ACM SIGMOD Record*, vol. 29, no. 2, pp. 1–12, 2000.

[6] W. Gan, J. C. W. Lin, P. Fournier-Viger, H. C. Chao, and J. Zhan, "Mining of frequent patterns with multiple minimum supports," *Engineering Applications of Artificial Intelligence*, vol. 60, pp. 83–96, 2017.

[7] Y. Shen, Z. Zhang, and Q. Yang, "Objective-oriented utility-based association mining," in *The International Conference on Data Mining*. IEEE, 2002, pp. 426–433.

[8] H. Yao, H. J. Hamilton, and C. J. Butz, "A foundational approach to mining itemset utilities from databases," in *The SIAM International Conference on Data Mining*. SIAM, 2004, pp. 482–486.

[9] W. Gan, J. C. W. Lin, P. Fournier-Viger, H. C. Chao, V. S. Tseng, and P. S. Yu, "A survey of utility-oriented pattern mining," *IEEE Transactions on Knowledge and Data Engineering*, vol. 33, no. 4, pp. 1306–1327, 2021.

[10] W. Gan, J. C. W. Lin, P. Fournier-Viger, H. C. Chao, T. P. Hong, and H. Fujita, "A survey of incremental high-utility itemset mining," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 8, no. 2, p. e1242, 2018.

[11] W. Gan, J. C. W. Lin, H. C. Chao, S. L. Wang, and P. S. Yu, "Privacy preserving utility mining: a survey," in *IEEE International Conference on Big Data*. IEEE, 2018, pp. 2617–2626.

[12] W. Gan, G. Chen, H. Yin, P. Fournier-Viger, C. M. Chen, and P. S. Yu, "Towards revenue maximization with popular and profitable products," *ACM/IMS Transactions on Data Science*, vol. 2, no. 4, pp. 1–21, 2022.

[13] R. Uday Kiran, T. Yashwanth Reddy, P. Fournier Viger, M. Toyoda, P. Krishna Reddy, and M. Kitsuregawa, "Efficiently finding high utility-frequent itemsets using cutoff and suffix utility," in *The Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 2019, pp. 191–203.

[14] J. Pillai and O. Vyas, "High utility rare item set mining (HURI): an approach for extracting high utility rare item sets," *Journal on Future Engineering and Technology*, vol. 7, no. 1, pp. 1–10, 2011.

[15] Y. Gui, W. Gan, Y. Chen, and Y. Wu, "Mining with rarity for web intelligence," in *Companion Proceedings of the Web Conference*, 2022, pp. 973–981.

[16] S. Wan, J. Chen, Z. Qi, W. Gan, and L. Tang, "Fast RFM model for customer segmentation," in *Companion Proceedings of the Web Conference*, 2022, pp. 965–972.

[17] W. Gan, L. Chen, S. Wan, J. Chen, and C. M. Chen, "Anomaly rule detection in sequence data," *IEEE Transactions on Knowledge and Data Engineering*, pp. 1–14, 2021.

[18] M. Iqbal, M. N. Setiawan, M. I. Irawan, K. M. N. K. Khalif, N. Muhammad, and B. M. Aziz, "Cardiovascular disease detection from high utility rare rule mining," *Artificial Intelligence in Medicine*, pp. 102 347–102 358, 2022.

[19] M. J. Zaki, "Scalable algorithms for association mining," *IEEE Transactions on Knowledge and Data Engineering*, vol. 12, no. 3, pp. 372–390, 2000.

[20] J. Pei, J. Han, and R. Mao, "CLOSET: An efficient algorithm for mining frequent closed itemsets," in *ACM SIGMOD Workshop*, vol. 4, 2000, pp. 21–30.

[21] T. Uno, M. Kiyomi, and H. Arimura, "LCM Ver. 2: Efficient mining algorithms for frequent/closed/maximal itemsets," in *The IEEE ICDM Workshop on Frequent Itemset Mining Implementations*, vol. 126, 2004, pp. 1–11.

[22] J. Pei, J. Han, H. Lu, S. Nishio, S. Tang, and D. Yang, "H-Mine: Fast and space-preserving frequent pattern mining in large databases," *IIE Transactions*, vol. 39, no. 6, pp. 593–605, 2007.

[23] L. Szathmary, A. Napoli, and P. Valtchev, "Towards rare itemset mining," in *The 19th IEEE international Conference on Tools with Artificial Intelligence*, vol. 1. IEEE, 2007, pp. 305–312.

[24] L. Troiano, G. Scibelli, and C. Birtolo, "A fast algorithm for mining rare itemsets," in *The 9th International Conference on Intelligent Systems Design and Applications*. IEEE, 2009, pp. 1149–1155.

[25] W. Gan, J. C. W. Lin, P. Fournier-Viger, H. C. Chao, J. Zhan, and J. Zhang, "Exploiting highly qualified pattern with frequency and weight occupancy," *Knowledge and Information Systems*, vol. 56, no. 1, pp. 165–196, 2018.

[26] Y. Liu, W. K. Liao, and A. Choudhary, "A two-phase algorithm for fast discovery of high utility itemsets," in *The Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 2005, pp. 689–695.

[27] V. S. Tseng, C. W. Wu, B. E. Shie, and P. S. Yu, "UP-Growth: an efficient algorithm for high utility itemset mining," in *The 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2010, pp. 253–262.

[28] M. Liu and J. Qu, "Mining high utility itemsets without candidate generation," in *The 21st ACM International Conference on Information and Knowledge Management*, 2012, pp. 55–64.

[29] P. Fournier Viger, C. W. Wu, S. Zida, and V. S. Tseng, "FHM: Faster high-utility itemset mining using estimated utility co-occurrence pruning," in *The International Symposium on Methodologies for Intelligent Systems*. Springer, 2014, pp. 83–92.

[30] S. Zida, P. Fournier Viger, J. C. W. Lin, C. W. Wu, and V. S. Tseng, "EFIM: a fast and memory efficient algorithm for high-utility itemset mining," *Knowledge and Information Systems*, vol. 51, no. 2, pp. 595–625, 2017.

[31] C. F. Ahmed, S. K. Tanbeer, B. S. Jeong, and Y. K. Lee, "Efficient tree structures for high utility pattern mining in incremental databases," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 12, pp. 1708–1721, 2009.

[32] J. C. W. Lin, T. P. Hong, and W. H. Lu, "An effective tree structure for mining high utility itemsets," *Expert Systems with Applications*, vol. 38, no. 6, pp. 7419–7424, 2011.

[33] J. Qu, M. Liu, and P. Fournier Viger, "Efficient algorithms for high utility itemset mining without candidate generation," in *High-Utility Pattern Mining*. Springer, 2019, pp. 131–160.

[34] W. Gan, J. C. W. Lin, P. Fournier-Viger, H. C. Chao, and P. S. Yu, "HUOPM: High-utility occupancy pattern mining," *IEEE Transactions on Cybernetics*, vol. 50, no. 3, pp. 1195–1208, 2020.

[35] S. Wan, W. Gan, X. Guo, J. Chen, and U. Yun, "FUIM: Fuzzy utility itemset mining," *arXiv preprint arXiv:2111.00307*, pp. 1–12, 2021.

[36] S. Wan, Z. Ye, W. Gan, and J. Chen, "Temporal fuzzy utility maximization with remaining measure," *arXiv preprint arXiv:2208.12439*, pp. 1–14, 2022.

[37] V. Goyal, S. Dawar, and A. Sureka, "High utility rare itemset mining over transaction databases," in *International Workshop on Databases in Networked Information Systems*. Springer, 2015, pp. 27–40.

[38] J. Miao, W. Gan, S. Wan, Y. Wu, and P. Fournier-Viger, "Towards target high-utility itemsets," *arXiv preprint arXiv:2206.06157*, pp. 1–12, 2022.

[39] J. Miao, S. Wan, W. Gan, J. Sun, and J. Chen, "Targeted high-utility itemset querying," *IEEE Transactions on Artificial Intelligence*, pp. 1–13, 2022.

[40] M. Kubat, A. Hafez, V. V. Raghavan, J. R. Lekkala, and W. K. Chen, "Itemset trees for targeted association querying," *IEEE Transactions on Knowledge and Data Engineering*, vol. 15, no. 6, pp. 1522–1534, 2003.

[41] P. Fournier-Viger, E. Mwamikazi, T. Gueniche, and U. Faghihi, "MEIT: Memory efficient itemset tree for targeted association rule mining," in *International Conference on Advanced Data Mining and Applications*. Springer, 2013, pp. 95–106.

[42] C. Chand, A. Thakkar, and A. Ganatra, "Target oriented sequential pattern mining using recency and monetary constraints," *International Journal of Computer Applications*, vol. 45, no. 10, pp. 12–18, 2012.

[43] C. Zhang, Z. Du, Q. Dai, W. Gan, J. Weng, and P. S. Yu, "TUSQ: Targeted high-utility sequence querying," *IEEE Transactions on Big Data*, pp. 1–16, 2022.

[44] R. Rymon, "Search through systematic set enumeration," in *The 3rd International Conference on Principles of Knowledge Representation and Reasoning*, 1992, pp. 539–550.

[45] S. Krishnamoorthy, "Pruning strategies for mining high utility itemsets," *Expert Systems with Applications*, vol. 42, no. 5, pp. 2371–2381, 2015.